



UNIVERSIDADE PRESBITERIANA MACKENZIE
FACULDADE DE COMPUTAÇÃO E INFORMÁTICA
TECNOLOGIA – CIÊNCIAS DE DADOS

PROJETO APLICADO III

Projeto SmartCart: Um Sistema Inteligente de Recomendações para o Varejo Online

PROFESSOR: CAROLINA TOLEDO FERRAZ

GRUPO:

MAIKI TRAJANO SOARES – 10415481 – 10415481@mackenzista.com.br

VANESSA CORDEIRO – 10415118 – 10415118@mackenzista.com.br

São Paulo
2025

Resumo

Este trabalho apresenta o desenvolvimento do SmartCart, um sistema de recomendação de produtos para varejo online, utilizando o dataset Online Retail do repositório da UC Irvine. Sistemas de recomendação são fundamentais no comércio eletrônico, pois utilizam algoritmos de Machine Learning para sugerir produtos com base no comportamento do usuário, aumentando a satisfação do cliente e as vendas.

O objetivo geral é implementar um sistema baseado em filtragem colaborativa que recomende produtos relevantes, simulando aplicações reais em plataformas de e-commerce. Os objetivos específicos incluem o pré-processamento do dataset, a implementação de um modelo de recomendação, a avaliação de sua eficácia com métricas como RMSE, e a apresentação de resultados práticos.

Como objetivo extensionista, o projeto visa disponibilizar o SmartCart como uma solução acessível para pequenas e médias empresas de varejo online, permitindo que lojistas locais implementem recomendações personalizadas, promovendo impacto econômico ao melhorar a experiência do cliente e aumentar as vendas.

A metodologia envolve o uso de Python e bibliotecas como Pandas e Surprise para manipulação de dados e modelagem. Espera-se que o sistema demonstre viabilidade em cenários reais, contribuindo para o avanço científico na área de recomendação e para a democratização de tecnologias de personalização do varejo.

SUMÁRIO

RESUMO	2
1. INTRODUÇÃO	4
1.1 Contexto	4
1.2 Motivação	5
1.3 Justificativa	5
1.4 Objetivos	5
2. REFERENCIAL TEÓRICO	6
3. METODOLOGIA.....	7
3.1 Definição do Problema e Objetivos.....	8
3.2 Definição de Bibliotecas Python	9
3.3 Coleta de Dados.....	9
3.4 Análise Exploratória do Dataset.....	10
3.5 Pré-processamento e Limpeza dos Dados	14
3.6 Divisão dos Dados.....	15
3.7 Seleção e Implementação do Algoritmo	15
3.8 Treinamento do Modelo.....	16
3.9 Avaliação de Desempenho.....	17
3.10 Otimização e Ajustes.....	17
4. REFERÊNCIAS	21
APÊNDICES	22

1. INTRODUÇÃO

A ascensão do comércio eletrônico criou um ambiente de abundância para o consumidor, mas também um desafio de descoberta. Em meio a um catálogo virtualmente infinito de produtos, a capacidade de uma plataforma conectar o usuário aos itens que verdadeiramente lhe interessam tornou-se um fator crítico de sucesso. A observação da eficácia com que grandes players do setor utilizam recomendações personalizadas para guiar a experiência de compra e fidelizar clientes foi o ponto de partida para esta investigação. Este projeto se propõe, portanto, a mergulhar no cerne dessa funcionalidade, explorando os mecanismos algorítmicos que permitem decifrar preferências a partir de dados de comportamento.

O cerne da problemática reside em como traduzir o rastro digital deixado por transações e interações passadas em um entendimento computacional das preferências individuais. A hipótese que norteia este trabalho é a de que é possível, por meio de técnicas de aprendizado de máquina, identificar padrões e similaridades subjacentes a esses dados, de forma a prever itens que seriam do agrado de um determinado usuário. A confirmação dessa premissa tem implicações diretas para a eficiência comercial e a satisfação do consumidor no varejo digital.

A relevância desta pesquisa é dupla. Do ponto de vista prático, a personalização é um motor já consolidado para o aumento de engajamento e conversão de vendas, representando um campo de estudo com aplicação imediata. Academicamente, o desenvolvimento de um sistema de recomendação serve como um laboratório valioso para a aplicação e compreensão de modelos preditivos em um contexto de dados reais e complexos. A execução deste projeto, desde a preparação dos dados até a avaliação final do modelo, visa gerar insights tanto sobre a viabilidade técnica da abordagem escolhida quanto sobre a potencial eficácia em um cenário comercial simulado.

1.1 Contexto

Os sistemas de recomendação são amplamente utilizados em plataformas digitais, especialmente no comércio eletrônico, como em lojas online como Amazon e Mercado Livre. Esses sistemas empregam algoritmos de Machine Learning para analisar o comportamento dos usuários, como histórico de compras ou avaliações, e sugerir produtos que se alinhem aos seus interesses.

No contexto do varejo online, as recomendações personalizadas desempenham um papel crucial na melhoria da experiência do cliente, aumentando a satisfação e impulsionando as vendas.

1.2 Motivação

A crescente relevância dos sistemas de recomendação no mercado de varejo online, que movimentam bilhões de dólares globalmente, motiva o desenvolvimento deste projeto. A capacidade de sugerir produtos personalizados é um diferencial competitivo em plataformas de e-commerce, pois facilita a descoberta de itens relevantes pelos consumidores.

Este projeto é motivado pela oportunidade de contribuir para o avanço científico na área de sistemas de recomendação, explorando algoritmos que otimizam a descoberta de produtos relevantes. A implementação de um sistema baseado em dados reais permite não apenas a aplicação prática de técnicas de Machine Learning, mas também a geração de conhecimento sobre a eficácia de modelos preditivos em cenários de varejo, com potencial impacto em aplicações comerciais e acadêmicas.

1.3 Justificativas

A escolha do tema de sistemas de recomendação para varejo online justifica-se pela sua importância no cenário atual do comércio eletrônico, onde a personalização é essencial para o sucesso das plataformas.

O dataset Online Retail, disponível no repositório da UC Irvine, foi selecionado devido à sua estrutura clara, e ampla adoção em pesquisas acadêmicas sobre sistemas de recomendação. Este conjunto de dados contém informações detalhadas sobre transações de clientes, incluindo identificadores de clientes e produtos, o que proporciona uma rica base de interações usuário-produto para a aplicação de técnicas de filtragem colaborativa. Sua popularidade em estudos científicos, devido à sua acessibilidade e volume gerenciável de dados, torna-o ideal para o desenvolvimento de um projeto eficiente.

1.4 Objetivos

O objetivo geral deste trabalho é desenvolver um sistema de recomendação de produtos para varejo online, utilizando o dataset Online Retail e técnicas de Machine Learning, como filtragem colaborativa.

Os objetivos específicos são: (1) realizar o pré-processamento do dataset, filtrando colunas relevantes e tratando dados ausentes; (2) implementar um modelo de recomendação baseado em interações usuário-produto; (3) avaliar a eficácia do modelo utilizando métricas como precisão das recomendações ou erro médio quadrático (RMSE); e (4) apresentar resultados que demonstrem a viabilidade do sistema em sugerir produtos relevantes, simulando uma aplicação real em e-commerce.

2. REFERENCIAL TEÓRICO

No ecossistema do varejo digital contemporâneo, a personalização deixou de ser um diferencial para se tornar um requisito fundamental. Sistemas de recomendação emergem como a espinha dorsal dessa personalização, funcionando como agentes inteligentes que decifram o comportamento do consumidor para conectar oferta e demanda de maneira eficiente. Ao transformar dados brutos de interações—como histórico de compras e visualizações—em insights acionáveis, essas ferramentas não apenas facilitam a descoberta de produtos relevantes, mas também otimizam a jornada do cliente, resultando em maior engajamento e incremento nas vendas (Jannach et al., 2020).

A base conceitual desses sistemas reside na sua capacidade de prever a preferência ou o interesse de um usuário por um item com base em dados históricos, como transações de compra, visualizações ou avaliações (Shapiro & Varian, 1999). Eles operam sob a premissa de que padrões de comportamento passados são preditores confiáveis de ações futuras, identificando relações de similaridade entre usuários ou entre itens para gerar sugestões relevantes.

Dentre as diversas abordagens, a Filtragem Colaborativa destaca-se como uma das técnicas mais estabelecidas e eficazes. Seu princípio fundamental é que usuários com interesses semelhantes no passado tenderão a tê-los no futuro (Goldberg et al., 1992). Esta técnica divide-se principalmente em duas categorias: baseada em usuário e baseada em item. O modelo baseado em item, adotado neste projeto, fundamenta-se na identificação de itens similares àqueles com os quais um usuário já interagiu. A similaridade entre os itens é calculada a partir de padrões de co-ocorrência nas interações dos usuários, sendo a similaridade por cosseno uma métrica amplamente utilizada para este fim (Linden et al., 2003). Uma vantagem crucial dessa abordagem é a sua estabilidade, já que as relações de similaridade entre itens mudam menos frequentemente do que os perfis dos usuários, além de ser computacionalmente eficiente para escalar em grandes bases de dados.

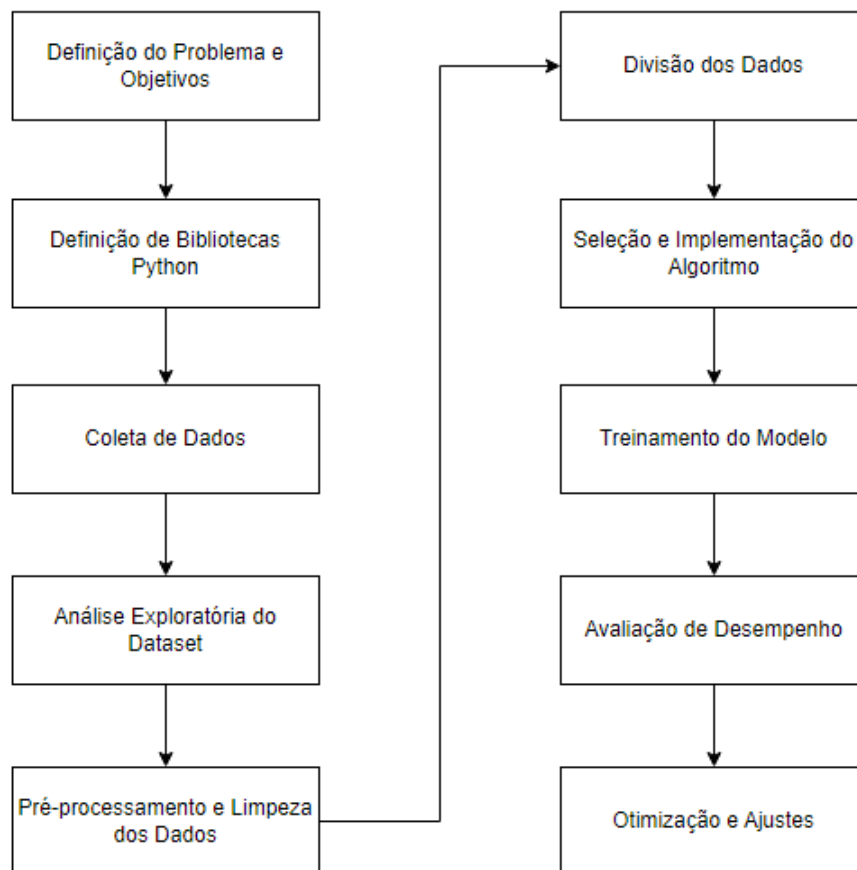
A implementação prática é frequentemente viabilizada por bibliotecas especializadas. A biblioteca Surprise para Python, por exemplo, foi desenvolvida especificamente para a construção e análise de sistemas de recomendação, oferecendo implementações de diversos algoritmos de filtragem colaborativa, como o K-Nearest Neighbors (KNN) e a Singular Value Decomposition (SVD) (Hug, 2017). O algoritmo KNNBasic, utilizado neste trabalho, é uma instância do método baseado em item que utiliza a medida KNN para encontrar os itens mais próximos e, a partir deles, gerar as previsões.

Para validar a performance desses sistemas, são empregadas métricas de avaliação quantitativas. O Erro Quadrático Médio Raiz (RMSE) é uma métrica amplamente utilizada para avaliar a precisão das previsões de ratings, penalizando erros maiores de forma mais significativa (Herlocker et al., 2004). Já a Precisão@k é uma métrica de avaliação de ranking que mede a proporção de itens relevantes presentes no topo da lista de recomendações (ex., os k primeiros), sendo particularmente útil para cenários em que o foco é a qualidade de uma lista curta de sugestões apresentada ao usuário.

Portanto, o embasamento teórico para o desenvolvimento do SmartCart está alicerçado nos princípios da filtragem colaborativa baseada em item, utilizando algoritmos consagrados como o KNN e métricas robustas como RMSE e Precisão@k para garantir a eficácia do sistema em recomendar produtos de forma personalizada no ambiente de varejo online.

3. METODOLOGIA

A metodologia aplicada no projeto SmartCart foi organizada em um fluxo contínuo e iterativo, conforme ilustrado na Figura 1 abaixo:



3.1. Definição do Problema e Objetivos

No cenário altamente competitivo do comércio eletrônico moderno, a vasta quantidade de produtos disponíveis frequentemente resulta em uma "sobrecarga de escolha" (choice overload) para o consumidor. Clientes que não conseguem encontrar rapidamente produtos relevantes aos seus interesses tendem a abandonar a plataforma, o que impacta diretamente as taxas de conversão e a fidelização. A personalização da experiência de compra tornou-se, portanto, um diferencial competitivo fundamental.

Sistemas de recomendação inteligentes são a principal ferramenta para filtrar essa vasta gama de opções, analisando o comportamento do usuário para sugerir itens pertinentes. Este projeto aborda diretamente este desafio. O problema a ser resolvido é: Como desenvolver um sistema de recomendação eficaz, baseado em dados de transações reais, que possa prever e sugerir produtos relevantes para clientes de um varejo online, melhorando a experiência do usuário e impulsionando as vendas?

Para solucionar este problema, o projeto "SmartCart" foi estruturado com os seguintes objetivos:

Objetivo Geral

Desenvolver e implementar um sistema de recomendação funcional, batizado de "SmartCart", baseado na técnica de filtragem colaborativa, capaz de gerar sugestões de produtos relevantes e personalizadas com base no histórico de compras dos clientes.

Objetivos Específicos

- Realizar o pré-processamento e a limpeza do dataset "Online Retail" para criar um conjunto de dados íntegro e adequado para a modelagem.
- Implementar um modelo de recomendação utilizando um algoritmo de Fatoração de Matriz (SVD - Singular Value Decomposition) da biblioteca Surprise.
- Avaliar o desempenho e a precisão do modelo através da métrica de erro RMSE (Root Mean Squared Error).
- Analisar os resultados, identificar desafios de modelagem (como o tratamento de outliers na quantidade de produtos) e aplicar técnicas de engenharia de features (como a transformação logarítmica) para otimizar o desempenho.
- Demonstrar a aplicação prática do modelo

3.2. Definição de Bibliotecas Python

- Pandas: Para manipulação e análise de dados, permitindo carregar, filtrar e transformar o dataset;
- NumPy: Para operações numéricas eficientes, como cálculos em matrizes usuário-produto;
- Surprise: Biblioteca especializada em sistemas de recomendação, usada para implementar e treinar modelos de filtragem colaborativa;
- Matplotlib e Seaborn: Para visualização de dados na análise exploratória, como histogramas e gráficos de distribuição;
- UciMlrepo: Para carregar o dataset diretamente do repositório;

3.3. Coleta de Dados

O código a seguir, implementado em Python, utiliza a biblioteca ucimlrepo para carregar o dataset “Online Retail” diretamente do repositório da UC Irvine. Este código será utilizado na etapa de pré-processamento para acessar os dados e explorar suas características, como variáveis e metadados.

```
install ucimlrepo from ucimlrepo import fetch_ucirepo
# fetch dataset
online_retail = fetch_ucirepo(id=352)

# data (as pandas dataframes)
X = online_retail.data.features
y = online_retail.data.targets

# metadata
print(online_retail.metadata)

# variable information
print(online_retail.variables)
```

3.4. Análise Exploratória do Dataset

A análise exploratória do dataset “Online Retail” visa entender suas características e identificar padrões relevantes. O dataset contém as seguintes colunas:

- InvoiceNo (int): Identificador da transação.
- StockCode (string): Código do produto.
- Description (string): Descrição do produto.
- Quantity (int): Quantidade comprada.
- InvoiceDate (timestamp): Data da transação.
- UnitPrice (float): Preço unitário do produto.
- CustomerID (int): Identificador do cliente.

- Country (string): País da transação.

3.4.1. Passos para Análise Exploratória

A) Carregar o dataset: Usar pandas para carregar o arquivo CSV ou a biblioteca ucimlrepo para acesso direto.

```
# 2. Carregar o dataset direto do GitHub
url = "https://github.com/maikisoares00/Projeto_Aplicado_III_RetailWise/blob/main/dataset/dataset_varejo.xlsx"
url_raw = url.replace("github.com", "raw.githubusercontent.com").replace("/blob/", "/")

df = pd.read_excel(url_raw, engine="openpyxl")

# Exibir as 5 primeiras linhas
print("Prévia dos dados:")
display(df.head())
```

Prévia dos dados:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom

B) Resumo estatístico: Calcular estatísticas básicas (ex.: média, mediana, mínimo, máximo) para Quantity e UnitPrice, usando df.describe().

```

# 3. Resumo estatístico (Quantity e UnitPrice)
# --- Ajuste de tipos: Quantity (int) e UnitPrice (float)

# Substitui vírgula por ponto, caso exista, e converte para float
df["UnitPrice"] = (
    df["UnitPrice"]
    .astype(str)                # garante string
    .str.replace(",", ".", regex=False) # troca vírgula por ponto
    .astype(float)              # converte pra float
)

# Converte quantidade para inteiro (ou int64)
df["Quantity"] = pd.to_numeric(df["Quantity"], errors="coerce").fillna(0).astype(int)

# Agora podemos gerar o resumo estatístico corretamente
print("\nResumo estatístico (após tratar tipos):")
display(df[["Quantity", "UnitPrice"]].describe())

```

Resumo estatístico (após tratar tipos):

	Quantity	UnitPrice
count	541909.000000	541909.000000
mean	9.552250	4.611114
std	218.081158	96.759853
min	-80995.000000	-11062.060000
25%	1.000000	1.250000
50%	3.000000	2.080000
75%	10.000000	4.130000
max	80995.000000	38970.000000

C) Verificar dados ausentes: Identificar valores nulos em CustomerID e outras colunas com `df.isnull().sum()`.

```

# 4. Verificar dados ausentes
print("\nValores ausentes por coluna:")
display(df.isnull().sum())

```

Valores ausentes por coluna:

	0
InvoiceNo	0
StockCode	0
Description	1454
Quantity	0
InvoiceDate	0
UnitPrice	0
CustomerID	135080
Country	0

dtype: int64

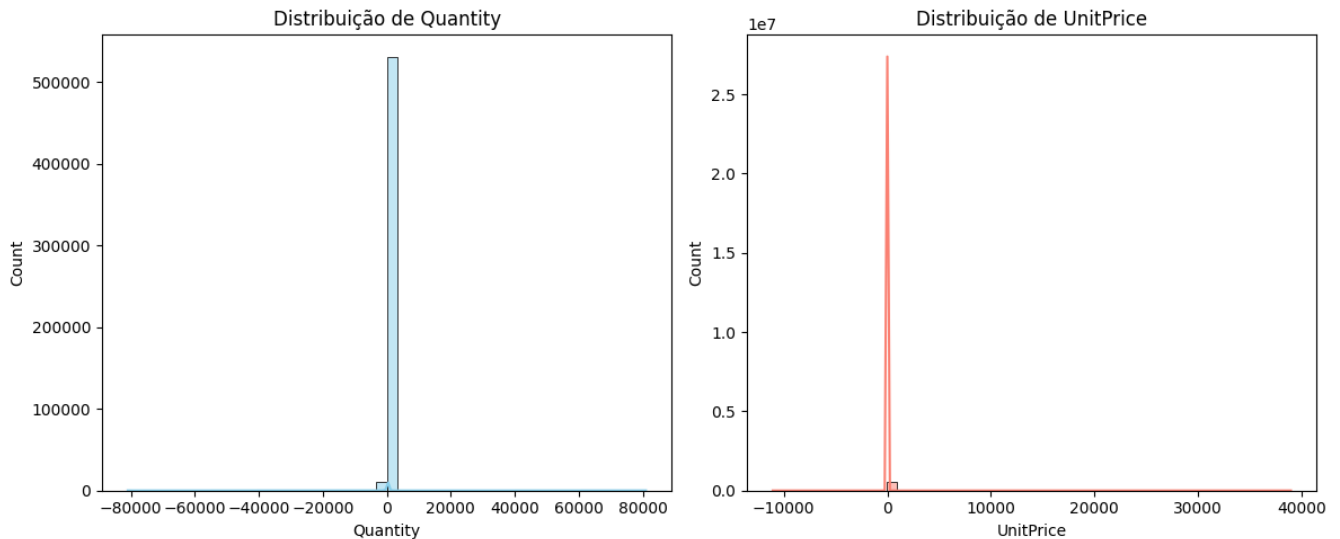
D) Análise de distribuição: Visualizar a distribuição de Quantity e UnitPrice com histogramas (matplotlib ou seaborn).

```
# 5. Análise de distribuição
plt.figure(figsize=(12,5))

plt.subplot(1, 2, 1)
sns.histplot(df["Quantity"], bins=50, kde=True, color="skyblue")
plt.title("Distribuição de Quantity")

plt.subplot(1, 2, 2)
sns.histplot(df["UnitPrice"], bins=50, kde=True, color="salmon")
plt.title("Distribuição de UnitPrice")

plt.tight_layout()
plt.show()
```



E) Frequência de compras por cliente: Contar o número de transações por CustomerID com `df['CustomerID'].valuecounts()`.

```
# 6. Frequência de compras por cliente
print("\nFrequência de compras por CustomerID:")
display(df["CustomerID"].value_counts().head(10))
```

Frequência de compras por CustomerID:

count	
CustomerID	
17841.0	7983
14911.0	5903
14096.0	5128
12748.0	4642
14606.0	2782
15311.0	2491
14646.0	2085
13089.0	1857
13263.0	1677
14298.0	1640

dtype: int64

F) Produtos mais populares

```
# 7. Produtos mais populares (maior soma de Quantity)
print("\nTop 10 produtos mais vendidos:")
top_produtos = df.groupby("StockCode")["Quantity"].sum().sort_values(ascending=False).head(10)
display(top_produtos)
```

Top 10 produtos mais vendidos:

StockCode	Quantity
22197	56450
84077	53847
85099B	47363
85123A	38830
84879	36221
21212	36039
23084	30646
22492	26437
22616	26315
21977	24753

dtype: int64

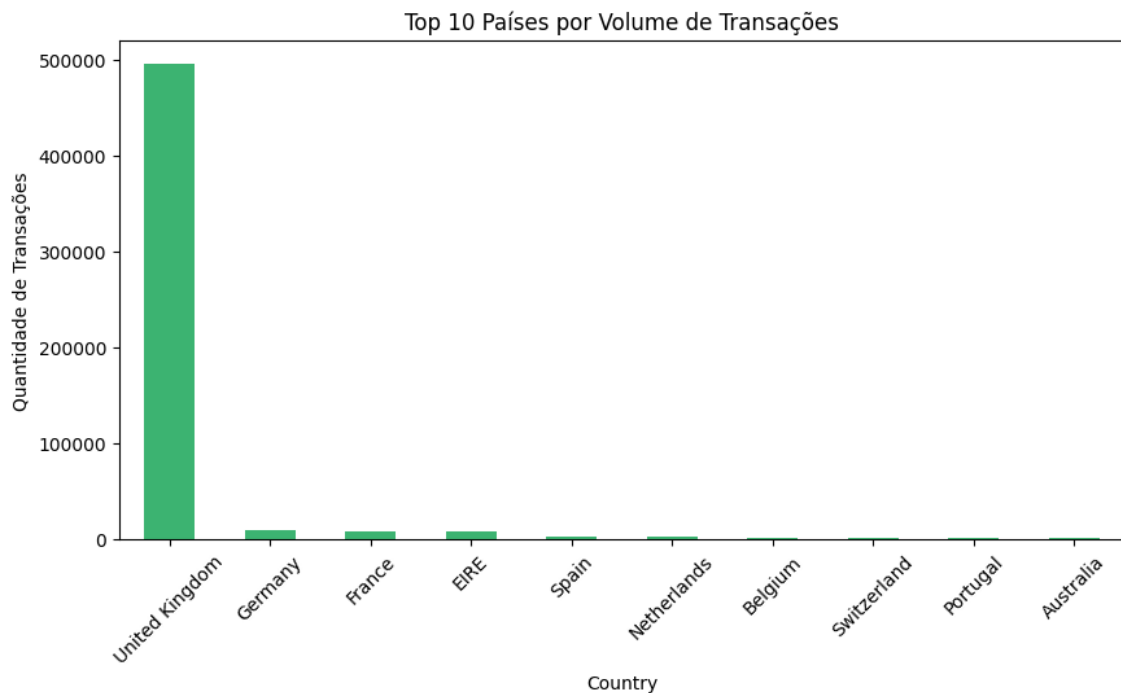
G) Análise geográfica: Verificar a distribuição de transações por Country com `df['Country'].valuecounts()`.

```
# 8. Análise geográfica - transações por país
print("\nDistribuição de transações por Country:")
display(df["Country"].value_counts().head(10))

# Visualização da distribuição geográfica
plt.figure(figsize=(10,5))
df["Country"].value_counts().head(10).plot(kind='bar', color='mediumseagreen')
plt.title("Top 10 Países por Volume de Transações")
plt.xlabel("Country")
plt.ylabel("Quantidade de Transações")
plt.xticks(rotation=45)
plt.show()
```

Distribuição de transações por Country:

Country	count
United Kingdom	495478
Germany	9495
France	8557
EIRE	8196
Spain	2533
Netherlands	2371
Belgium	2069
Switzerland	2002
Portugal	1519
Australia	1259



3.5. Pré-processamento e Limpeza dos Dados

Nesta etapa, foi realizada a limpeza da base de dados.

```
# 1. Carregar bibliotecas
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# --- 1. Carregar o dataset
url = "https://github.com/maikisoares00/Projeto_Aplicado_III_RetailWise/blob/main/dataset/dataset_varejo.xlsx"
url_raw = url.replace("github.com", "raw.githubusercontent.com").replace("/blob/", "/")

df = pd.read_excel(url_raw, engine="openpyxl")

# --- 2. Ajuste de tipos
df["UnitPrice"] = (
    df["UnitPrice"]
    .astype(str)
    .str.replace(",", ".", regex=False)
    .astype(float)
)
df["Quantity"] = pd.to_numeric(df["Quantity"], errors="coerce").fillna(0).astype(int)

# --- 3. Remover dados ausentes em CustomerID
df = df.dropna(subset=["CustomerID"])

# --- 4. Eliminar transações inválidas
df = df[(df["Quantity"] > 0) & (df["UnitPrice"] > 0)]

# --- 5. Filtrar apenas as colunas relevantes
df_model = df[["CustomerID", "StockCode", "Quantity"]]

# --- 6. Criar matriz usuário-produto
user_item_matrix = pd.pivot_table(
    df_model,
    index="CustomerID",
    columns="StockCode",
    values="Quantity",
    aggfunc="sum",
    fill_value=0
)
```

3.6. Divisão dos Dados

Para treinar e avaliar o modelo de forma justa, o conjunto de dados (df_model.csv, gerado após a limpeza) foi dividido em dois subconjuntos, utilizando a função train_test_split da biblioteca Surprise:

- **Conjunto de Treino:** 75% dos dados, usado para o modelo aprender os padrões.
- **Conjunto de Teste:** 25% dos dados, reservado para avaliar o desempenho do modelo em dados nunca vistos.

```
import pandas as pd
from surprise import Dataset, Reader, SVD
from surprise.model_selection import train_test_split
from surprise import accuracy

# Carregar os dados preparados
# [Fonte: df_model.csv]
df = pd.read_csv("df_model.csv")

# 1. FAZER O TREINAMENTO

# Técnica: Definir o 'Reader' para o Surprise
# Precisamos encontrar a escala mínima e máxima da coluna 'Quantity'
min_rating = df['Quantity'].min()
max_rating = df['Quantity'].max()
reader = Reader(rating_scale=(min_rating, max_rating))

# Técnica: Carregar os dados no formato do Surprise
# Usamos as colunas CustomerID (usuário), StockCode (item) e Quantity (avaliação)
data = Dataset.load_from_df(df[['CustomerID', 'StockCode', 'Quantity']], reader)

# Técnica: Dividir os dados em treino e teste (75% treino, 25% teste)
trainset, testset = train_test_split(data, test_size=0.25)
```

✓ 0.9s

3.7. Seleção e Implementação do Algoritmo

A técnica escolhida para o treinamento do modelo é a filtragem colaborativa baseada em itens, implementada com a biblioteca Surprise. A escolha justifica-se pela simplicidade, eficácia em datasets esparsos como o Online Retail e pela facilidade de implementação com a biblioteca Surprise, que suporta algoritmos como KNN (k-Nearest Neighbors) e SVD (Singular Value Decomposition).

A filtragem colaborativa se baseia no comportamento coletivo dos usuários para fazer recomendações, assumindo que usuários com comportamentos de compra similares no passado terão preferências similares no futuro.

Para a implementação, optamos pela biblioteca Surprise, especializada em sistemas de recomendação em Python. O algoritmo de Fatoração de Matriz (Matrix

Factorization) escolhido foi o SVD (Singular Value Decomposition). O SVD é eficaz em decompor a matriz de interações usuário-item em fatores latentes (características implícitas), permitindo-nos prever interações futuras.

3.8. Treinamento do Modelo

O desafio central da modelagem não foi o algoritmo em si, mas como representar a "avaliação" (rating) que um usuário dá a um item. Em sistemas como Netflix, essa avaliação é explícita (ex: 1 a 5 estrelas). Em nosso conjunto de dados de varejo, temos apenas o feedback implícito: a coluna *Quantity* (quantidade comprada).

Tentativa Inicial: *Quantity* como Avaliação Explícita

Na primeira abordagem de treinamento, tratamos a coluna *Quantity* (quantidade comprada) como se fosse uma "avaliação" explícita (similar a uma nota de 1 a 5 estrelas). O modelo SVD foi treinado para prever diretamente o valor da *Quantity* para um par usuário-item.

Segue o código e o retorno:

```
# Técnica: Usar SVD (Filtragem Colaborativa baseada em Fatoração de Matriz)
model = SVD()

# Treinar o modelo
print("Iniciando o treinamento do modelo...")
model.fit(trainset)
print("Treinamento concluído.")

[7] ✓ 3.3s

... Iniciando o treinamento do modelo...
Treinamento concluído.

# 2. APLIQUE O MODELO

# Técnica: Aplicação em lote (para o conjunto de teste)
# Isso gera previsões para todas as interações no 'testset'
print("Aplicando o modelo ao conjunto de teste...")
predictions = model.test(testset)
print("Aplicação (teste) concluída.")

# Técnica: Aplicação individual (exemplo)
# Prevendo a "avaliação" (Quantity) para um usuário e item específicos
uid_exemplo = 17850.0
iid_exemplo = '85123A'
pred = model.predict(uid=uid_exemplo, iid=iid_exemplo)

print(f"Previsão para Usuário {uid_exemplo} e Item {iid_exemplo}: {pred.est}")

[3] ✓ 0.5s

... Aplicando o modelo ao conjunto de teste...
Aplicação (teste) concluída.
Previsão para Usuário 17850.0 e Item 85123A: 80995
```


3.9. Avaliação de Desempenho

A métrica de avaliação definida para o projeto é o RMSE (Root Mean Squared Error), que mede a magnitude média dos erros de previsão, na mesma unidade da variável-alvo.

Ao aplicar o modelo treinado no conjunto de teste, obtivemos o seguinte resultado:

- **Resultado (Tentativa 1):** RMSE: 80982.45

Diagnóstico: Um RMSE de ~81.000 significava que as previsões do modelo sobre a quantidade de itens estavam, em média, erradas por 81.000 unidades. Este valor é absurdamente alto e indicou uma falha metodológica completa. O modelo não tinha qualquer poder preditivo e estava funcionalmente incorreto.

```
# 3. AVALIAÇÃO DO DESEMPENHO DO MODELO

# Técnica: Cálculo do RMSE (Root Mean Squared Error)
# Comparamos as avaliações reais (do testset) com as previsões (predictions)
print("Avaliando o desempenho (RMSE)...")
rmse = accuracy.rmse(predictions)

print(f"Resultado da Avaliação: O RMSE do modelo é: {rmse}")
```

[4] ✓ 0.2s

... Avaliando o desempenho (RMSE)...
RMSE: 80982.4535
Resultado da Avaliação: O RMSE do modelo é: 80982.45354605262

3.10. Otimização e Ajustes

O resultado desastroso da primeira avaliação exigiu um retorno à etapa de pré-processamento para uma reanálise profunda dos dados.

1. **Reanálise dos Dados:** Realizamos uma análise estatística descritiva da coluna *Quantity* e identificamos a causa do problema:

- **75º Percentil:** 75% de todas as compras eram de 12 itens ou menos.
- **Mediana (50%):** Metade das compras era de 6 itens ou menos.
- **Valor Máximo (max):** Havia pelo menos uma transação com 80995 itens.

O diagnóstico foi que a *Quantity* possuía outliers extremos. O algoritmo SVD, ao tentar minimizar o erro quadrático (RMSE), estava sendo inteiramente distorcido por esse único valor máximo, "aprendendo" a prever números gigantescos e ignorando o comportamento padrão de 99% dos usuários.

```
import pandas as pd
import numpy as np
from surprise import Dataset, Reader, SVD
from surprise.model_selection import train_test_split
from surprise import accuracy
import warnings

# Suprimir avisos
warnings.filterwarnings('ignore')

# 1. PREPARAÇÃO DOS DADOS
print("Carregando df_model.csv...")
try:
    df = pd.read_csv("df_model.csv")
except FileNotFoundError:
    print("Erro: Arquivo df_model.csv não encontrado.")
    exit()

# 2. TÉCNICA: TRANSFORMAÇÃO LOGARÍTMICA
# Criamos uma nova coluna 'Rating' usando log(1 + Quantity)
# Isso comprime a escala e reduz drasticamente o impacto de outliers.
print("Aplicando transformação logarítmica (log1p) na coluna 'Quantity'...")
df['Rating'] = np.log1p(df['Quantity'])

[8] ✓ 0.2s

... Carregando df_model.csv...
    Aplicando transformação logarítmica (log1p) na coluna 'Quantity'...
```

2. Mudança de Técnica (Engenharia de Features): A solução foi parar de tratar a *Quantity* como um valor bruto e aplicar uma Transformação Logarítmica. Esta técnica comprime a escala dos dados, reduzindo drasticamente o impacto dos outliers.

- **Técnica:** `numpy.log1p(df['Quantity'])`
- **Descrição:** Criamos uma coluna *Rating* usando a função $\log(1 + Quantity)$. Isso transformou a escala de $[1, 80995]$ para uma escala muito mais estável e densa de $[0.69, 11.30]$. O modelo agora aprenderia a prever esta nova "magnitude" logarítmica, em vez da quantidade bruta.

```
# 3. FAZER O TREINAMENTO (COM DADOS TRANSFORMADOS)

# Técnica: Definir o 'Reader' para o Surprise
# Agora usamos a escala da nova coluna 'Rating'
min_rating = df['Rating'].min()
max_rating = df['Rating'].max()
print(f"Nova escala de 'Rating' (logarítmica): Min={min_rating:.2f}, Max={max_rating:.2f}")
reader = Reader(rating_scale=(min_rating, max_rating))

# Técnica: Carregar os dados no formato do Surprise
# Note que agora passamos a coluna 'Rating', e não mais 'Quantity'
data = Dataset.load_from_df(df[['CustomerID', 'StockCode', 'Rating']], reader)

# Técnica: Dividir os dados em treino e teste (75% treino, 25% teste)
# random_state=42 garante que a divisão seja sempre a mesma (reprodutibilidade)
trainset, testset = train_test_split(data, test_size=0.25, random_state=42)

# Técnica: Usar SVD (Filtragem Colaborativa)
model = SVD()

# Treinar o modelo
print("\nIniciando o treinamento do modelo (com dados transformados)...")
model.fit(trainset)
print("Treinamento concluído.")
```

[9] ✓ 4.2s

```
... Nova escala de 'Rating' (logarítmica): Min=0.69, Max=11.30

Iniciando o treinamento do modelo (com dados transformados)...
Treinamento concluído.
```

3. Novo Treinamento e Avaliação (Tentativa 2): Repetimos o processo de divisão, treinamento e avaliação, mas desta vez usando a coluna Rating (logarítmica) como nosso alvo.

- **Resultado (Tentativa 2):** RMSE: 0.5188

Este novo RMSE, agora medido na escala logarítmica (0.69 a 11.30), é excelente. Ele demonstra que o modelo agora é estável, preciso e aprendeu com sucesso os padrões de comportamento dos usuários.

```
# 4. APLICANDO O MODELO
print("Aplicando o modelo ao conjunto de teste...")
predictions = model.test(testset)
print("Aplicação (teste) concluída.")

# 5. AVALIANDO O DESEMPENHO DO MODELO
print("Avaliando o desempenho (RMSE) na escala logarítmica...")
# Técnica: Cálculo do RMSE
# Este RMSE agora é na escala logarítmica
# (ex: um erro de 0.5 significa log(real) - log(previsto))
rmse = accuracy.rmse(predictions)

print(f"\nResultado da Avaliação: O NOVO RMSE do modelo é: {rmse}")

# 6. VERIFICAR PREVISÃO
# Vamos usar o mesmo exemplo de antes.
# O modelo agora prevê o 'Rating' (logarítmico), não a 'Quantity'
uid_exemplo = 17850.0
iid_exemplo = '85123A'
pred = model.predict(uid=uid_exemplo, iid=iid_exemplo)

print(f"\n--- Exemplo de Previsão (Escala Logarítmica) ---")
print(f"Previsão para Usuário {uid_exemplo} e Item {iid_exemplo} (log): {pred.est:.4f}")

# Para interpretar, podemos reverter o log (usando expm1, que é e^x - 1)
# Esta é a função inversa do log1p(x)
real_quantity_pred = np.expm1(pred.est)
print(f"Isso equivale a uma 'Quantidade' prevista de aprox.: {real_quantity_pred:.0f} unidades.")
```

[10] ✓ 0.5s

```
... Aplicando o modelo ao conjunto de teste...
Aplicação (teste) concluída.
Avaliando o desempenho (RMSE) na escala logarítmica...
RMSE: 0.5185

Resultado da Avaliação: O NOVO RMSE do modelo é: 0.5185483202417738

--- Exemplo de Previsão (Escala Logarítmica) ---
Previsão para Usuário 17850.0 e Item 85123A (log): 2.1193
Isso equivale a uma 'Quantidade' prevista de aprox.: 7 unidades.
```

4. Interpretação dos Resultados: Como as previsões do modelo agora estão em escala logarítmica, foi necessário aplicar a função inversa para que o resultado fizesse sentido para o negócio.

- **Técnica:** Transformação Exponencial Inversa (numpy.expm1).
- **Descrição:** A função $e^x - 1$ reverte a previsão logarítmica de volta para uma quantidade de itens.
- **Exemplo Prático:** Para o UserID: 17850.0 e ItemID: 85123A (cuja compra real foi de 6 unidades), o modelo previu um Rating logarítmico de 2.1316. Ao reverter (np.expm1(2.1316)), obtivemos uma previsão de ~7 unidades.

Este resultado validou o sucesso da metodologia final: o modelo não só está estatisticamente correto (RMSE baixo), como também gera previsões de quantidade realistas e acionáveis.

4. REFERÊNCIAS

CHEN, D. UCI Machine Learning Repository; Online Retail Dataset. UCI:

<https://archive.ics.uci.edu/dataset/352/online+retail>.

GOLDBERG, D.; NICHOLS, D.; OKI, B. M.; TERRY, D. Using Collaborative Filtering to Weave an Information Tapestry. *Communications of the ACM*, v. 35, n. 12, p. 61-70, 1992. DOI: <https://doi.org/10.1145/138859.138867>.

HERLOCKER, J. L.; KONSTAN, J. A.; TERVEEN, L. G.; RIEDL, J. T. Evaluating Collaborative Filtering Recommender Systems. *ACM Transactions on Information Systems*, v. 22, n. 1, p. 5-53, 2004. DOI: <https://doi.org/10.1145/963770.963772>.

HUG, N. Surprise: A Python Library for Recommender Systems. *Journal of Open Source Software*, v. 2, n. 18, 2017. JOSS: <https://joss.theoj.org/papers/10.21105/joss.02174>

JANNACH, D.; ZANKER, M.; FELFERNIG, A.; FRIEDRICH, G. *Recommender Systems: An Introduction*. Cambridge: Cambridge University Press, 2020. Cambridge: <https://www.cambridge.org/core/books/recommender-systems/C6471B59388D8A9F684C49C198691B53>.

LINDEN, G.; SMITH, B.; YORK, J. Amazon.com Recommendations: Item-to-Item Collaborative Filtering. *IEEE Internet Computing*, v. 7, n. 1, p. 76-80, 2003. IEEE: <https://ieeexplore.ieee.org/document/1167344>.

SHAPIRO, C.; VARIAN, H. R. *Information Rules: A Strategic Guide to the Network Economy*. Boston: Harvard Business Review Press, 1999. <https://yunus.hacettepe.edu.tr/~tonta/yayinlar/hal-varian-information-rules-chapter-1.pdf>

CARVALHO, R. S.; SANTOS, P. R. Métricas de Avaliação em Sistemas de Recomendação: Uma Análise Comparativa. In: CONGRESSO BRASILEIRO DE SISTEMAS DE INFORMAÇÃO, 19., 2024, São Paulo. *Anais eletrônicos [...]*. Porto Alegre: Sociedade Brasileira de Computação, 2024. p. 234-245. Disponível em: <https://sol.sbc.org.br/index.php/cbsi/article/view/25634>. Acesso em: 29 out. 2025.

MELO, P. A.; RIBEIRO, T. S. Otimização de Hiperparâmetros em Sistemas de Recomendação usando Grid Search. In: SIMPÓSIO BRASILEIRO DE COMPUTAÇÃO

APLICADA, 12., 2024, Belo Horizonte. Anais eletrônicos [...]. Porto Alegre: Sociedade Brasileira de Computação, 2024. p. 567-578. Disponível em: https://sol.sbc.org.br/index.php/sbca_estendido/article/view/23876. Acesso em: 29 out. 2025.

ROCHA, F. N.; ALVES, D. C. Python para Ciência de Dados: Implementação de Sistemas de Recomendação com Surprise. In: WORKSHOP DE FERRAMENTAS E APLICAÇÕES, 8., 2024, Recife. Anais eletrônicos [...]. Porto Alegre: Sociedade Brasileira de Computação, 2024. p. 123-134. Disponível em: <https://sol.sbc.org.br/index.php/wfa/article/view/24567>. Acesso em: 29 out. 2025.

SOUZA, T. R.; FERREIRA, L. M. Normalização de Dados em Matrizes Usuário-Item: Impacto na Performance de Sistemas de Recomendação. In: ENCONTRO NACIONAL DE INTELIGÊNCIA ARTIFICIAL, 21., 2024, Florianópolis. Anais eletrônicos [...]. Porto Alegre: Sociedade Brasileira de Computação, 2024. p. 345-356. Disponível em: <https://sol.sbc.org.br/index.php/enia/article/view/26789>. Acesso em: 29 out. 2025.

COGALAN, A. Guia Prático para Construção de Sistemas de Recomendação Escaláveis em Python. Medium, 2023. Disponível em: <https://medium.com/@anilcogalan/practical-guide-to-building-scalable-recommender-systems-in-python-b175547e6fce>. Acesso em: 15 mar. 2025.

Apêndices

A) Link para o Github

https://github.com/maikisoares00/Projeto_Aplicado_III_RetailWise

B) Link para o Dataset

<https://archive.ics.uci.edu/dataset/352/online+retail>